



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

Project Report

SMART TRAFFIC CONTROLLER



Submitted by:

Akshit Mathur (23070123014)

Anupreeya Bhattacharyajee (23070123022)

Aryaman Pathak (23070123031)

Kautik Verma (23070123073)

Under the guidance of:

Dr. Shilpa Hudnurkar

Department of E&TC Engineering

Symbiosis Institute of Technology, Pune

Department of Electronics & Telecommunication



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

DECLARATION

We hereby declare that the project work entitled Smart Traffic Controller is an authentic record of our own work carried out as requirement of Mini project for the course of Microcontrollers and Applications of T.Y B.Tech Electronics & Telecommunication Engineering, Symbiosis Institute of Technology, Pune, under the guidance of Dr. Shilpa Hudnurkar, during July–December 2025.

Akshit Mathur (23070123014)

Anupreeya Bhattacharyajee (23070123022)

Aryaman Pathak (23070123031)

Kautik Verma (23070123073)

Date: 16/10/2025

Certified that the above statement made by the student is correct to the best of our knowledge and belief.

Dr. Shilpa Hudnurkar
Assistant Professor,
E&TC Dept, SIT, Pune



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

INDEX

Sr. No.	Content
1	Abstract
2	Block Diagram
3	Selection of Components
4	List of Components
5	Component Description
6	Code
7	Working
9	Problems Faced / Troubleshooting
10	Application of the Project
11	Conclusion
12	Future Scope of Work
13	References
14	Appendix



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

ABSTRACT

Traffic Congestion is one of the most major problems faced in each metropolitan cities due to increasing number of vehicles and inefficient traffic signal systems. Traditional Traffic Light Signal systems work on fixed cycles which is leading to unnecessary wasting time and fuel wastage even when roads are empty. To overcome from this problem, this project presents the design and implementation of Smart Traffic Controller using PIC18F4550 and Raspberry Pi for intelligent traffic management based on real-time vehicle density detection.

The highly specialized microcontrollers used in the system are a Raspberry Pi and a PIC18F4550 which work alongside each other. Its hardware consists of the Raspberry Pi, which acts as the central processing unit, responsible to do all the computation and logic, a Pi Camera attached to a servo motor to capture images, and some traffic light LEDs, attached to the Pi by its GPIO pins. The PIC18F4550 is entirely devoted to the accurate control of the servo motors, the camera is placed in three fixed positions (0°, 90° and 180°) according to the data received by the IR sensors. It uses separate internal communication standards: PIC uses PWM to signal to the servos, and Raspberry Pi uses its CSI and GPIO connections to the camera and direct control of the light, respectively.

The expected result of this project is a demonstrably reliable and independent system of traffic control operating on a feedback loop: the PIC will regulate the scan of the camera; the Raspberry Pi will calculate the density based on the images and future traffic signal timing will be determined by the Pi. With the green signal always given more time to the road that has the greatest number of vehicles, the system will result in a high and quantifiable cut in vehicle queue duration and congestion in general, and as a result make the urban transportation network efficient.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Introduction

Rapid increase in population in metropolitan cities increases the number of vehicles, managing the traffic congestion efficiently is one of the major problems. Traditional traffic lights work on fixed time duration providing same green lights to all the roads irrespective of their actual vehicle density. This results in several problems such as unnecessary waiting time, fuel wastages which increases air pollution and reduced traffic flow efficiency. Therefore, there is a need for intelligent system that can adjust the signal timing in real time conditions.

This project would design traffic management system which utilizes sensor detection and image-based vehicle counting to attain adaptive traffic control. This system is implemented on four-way junction provided infrared (IR) sensors on each road used to detect the presence of vehicle on each road. When heavy congestion is detected servo motor-controlled camera captures the image of that road. the Raspberry pi with OpenCV image processing analyse the captured frame to calculate vehicle density and results in improved green light duration for that road.

The solution suggested is a modular architecture with two specialized control units to provide high performance as well as reliability in its functioning. The Raspberry Pi is the central processing unit, as well as the brain of the system, and performs all the functionalities associated with the image acquisition via a Pi Camera, processing the acquired image with the help of OpenCV and machine learning algorithms to identify densities, and directly operates the traffic lights using its GPIO pins. In addition to the Pi, there is another PIC18F4550 microcontroller that is specifically used as a dedicated servo motor controller. This departmentalization leaves the PIC with the responsibility of ensuring an accurate location of the camera, rotating it to the three predetermined angles, namely, 0, 90, and 180 degrees, which are the intersection roads. The main hardware needed to make this work is the Raspberry Pi, the Pi camera, the PIC18F4550, one servo motor and some IR sensors to provide positional feedback.

By integrating this embedded system control with computer vision, the system effectively reduce traffic delays, reduces fuel consumption ensures smooth traffic movement. The project shows the collaborative model of microcontroller and image processing to develop scalable and intelligent traffic management solution.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Selection of Components:

1. PIC18F4550 Development Kit



2. Raspberry Pi



3. Infrared (IR) Sensors





SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

4. Pi Camera Module



5. Servo Motor



6. LEDs Red, Yellow, Green





SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

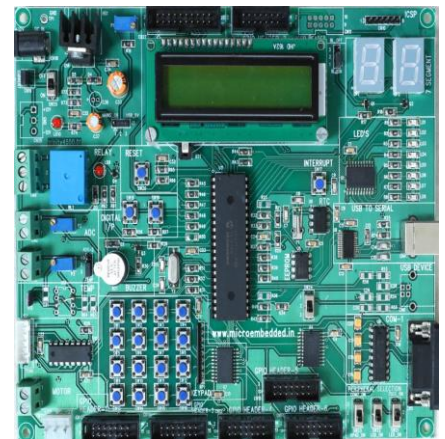
List of Components:

Name	Quantity
PIC18F4550 Development Kit	1
Raspberry Pi	1
Pi Camera Module V2	1
Servo Motor	1
Infrared (IR) Sensors	4
LEDs (Red,Yellow,Green)	12 (each LEDs 3)
USB cable	1
PCB/Breadboard	1

Components Description:

1. PIC18F4550 Microcontroller –

PIC18F4550 is an advance 8-bit microcontroller developed by microchip technology. It operates on 2V to 5V and supports maximum clock Frequency of 48MHz, offers 32KB of flash memory, 2KB of SRAM and 256 bytes of EEPROM and 13 channels of 10-bit ADC, Two CCP modules of PWM generation, Timers and build in support for UART, SPI, I2C Communication Protocol. In this Project PIC18F4550 act as main control unit reads the IR Sensors, Controlling Traffic Light Signals, Generating PWM signals for servo motor for camera positioning, Communicating Raspberry Pi with UART to receive improved signal timings.



- ##### **2. Raspberry Pi –**
- Raspberry Pi serves intelligent small processing Unit of the project. It is powered by 1.5GHz quad ARM Cortex A72 CPU and supports 4GB RAM, making suitable for real-time image processing task. In this system, Raspberry Pi captures images using Pi Camera Module Processes it using OpenCV to count vehicles within





SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

region of interest and calculates traffic density and computes the improved green light duration and transmits it to PIC18F4550 via UART Communication protocol.

3. **Pi Camera Module V2** – Pi Camera Module is an 8MP Camera used to provide live images for vehicle detection. It connects it with Raspberry Pi with CSI (Camera Serial Interface) and captures high quality images and videos on junction in real time. The capture Frames are processed using OpenCV to detect vehicles and count accurately. It enables the systems to adjust the signal timings according to traffic density.
4. **IR Sensors** – IR sensors used to detect the presence of vehicles on each road. Each Sensor comprises an IR transmitter (LED) and IR receiver (photodiode) that operates on 5V. When the vehicle obstructs the beam, the receiver output switches state from high to low indicating the presence of vehicle The sensor output is connected to RA0 – RA3 pins of PIC18F4550 configure as digital inputs a debounce delay ensures the stability in detection. This quickly senses the vehicle present on road support Raspberry pi camera for instant detection.
5. **Servo Motor** – Servo Motor is used to give the direction control for camera module. It operates on 4.8V to 6V and is driven by PWM signal from the CCP1 RC2 of PIC18F4550. The servo motor is controlled atb 50Hz with duty cyle of 1 to 2ms corresponding to 0°-180° rotation by mapping these angels to road servo motor orients the camera with high traffic density.



Working:

The project is an intelligent traffic monitoring and control that is based on a Raspberry Pi, a Pi camera, and PIC18F4550 microcontroller. Its primary objective is to minimize the traffic at intersections by monitoring traffic density and changing the signal timings. The vehicle and pedestrian separation are also within the system and this guarantees that the green light is distributed efficiently to the lanes that are most congested.

The microcontroller that is used as a servo motor controller is PIC18F4550. It is tasked with turning the Pi Camera in three fixed directions i.e. 0°, 90°, 180° directions which represent three roads upon crossing. In order to do so, three infrared (IR) sensors are linked to the PIC microcontroller, and each of them corresponds to one of the positions. Once a sensor of the IR is triggered, the microcontroller compares the sensor by producing a pulse-width modulated (PWM) signal to move the servo motor to the angle. The PIC also controls



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

a small LCD which shows the position of the camera at the current moment. Because the communication between the Raspberry Pi and the PIC18F4550 can be complicated directly, the PIC is employed as an independent servo controller with a specific task of the exact movement of the camera.

The Raspberry Pi is the brain of the system, and it is responsible not only in image processing but also in controlling traffic lights. Mounted on the servo motor is the Pi Camera, which takes pictures in every of the three directions as the camera turns. These pictures are analysed by the computer vision methods that are carried out in the form of OpenCV and machine learning algorithms. Each captured frame is analysed by Raspberry Pi in order to recognize the objects on the scene and differentiate between cars and people. Cars, trucks and motorcycles are counted to determine the density of traffic on a particular road whilst the pedestrians are not counted in this process. This difference is made because the green signals are not given to empty lanes or pedestrian lanes that do not need them.

After estimating the density of all the three roads using the Raspberry Pi, it identifies the lane that has the most vehicle concentration. The Raspberry Pi is then linked to the traffic lights which are then controlled in line with its GPIO pins. The road that has maximum number of vehicles is awarded with a green signal which is longer in duration compared to the other roads which are still red. Following a single signal cycle, the camera is then scanned in all the three directions again and the cycle is repeated. It is a self-corrective system that enables the system to real-time adapt itself to the evolving traffic conditions.

In general, the functioning of the system can be described in the following way: the servo motors movement which moves the Pi Camera to every road are controlled by the PIC18F4550 and the camera reaches images which are analysed to identify the vehicle density and a direct control over the traffic lights is provided by the Raspberry Pi. The system enables the reduction of waiting time of the lanes with the least congestion and efficient utilization of signal timing by prioritizing the road that has the highest congestion. The system is flexible, modular and could operate without human control, which is a result of the integration of computer vision and servo-controlled camera control.

Simulation:

We conducted a simulation to ensure every component of the system performed as intended before constructing the actual set up. The initial phase was devoted to the testing of the control of the servo motor and the IR sensor under the PIC18F4550. The simulation of the circuit was done using MPLAB X IDE software. The servo motor motion was tested virtually to determine whether it was able to move appropriately



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

to 0, 90 and 180 degrees as IR sensor of interest was activated. The LCD display was also to be tested to ensure that it displayed the position of the camera correctly each time the servo had moved.

Our small Python code with the OpenCV library processed the images and detected cars. We used three LEDs (red, yellow, and green) to replicate the functions of the traffic lights using the three GPIO of the Raspberry Pi. On executing the detection code, we were able to observe the lights being altered according to the number of vehicles coming in each direction. This proved that we were making the right decision by prioritizing the busiest lane.

These simulations made us know how to time things, what delays, and what interacts with what. This allowed us to test all the details in a virtual environment and eliminate minor bugs beforehand and ensure that when we finally transitioned to hardware everything was going to work.

Problems Faced:

1) Logic Voltage Mismatch:

Our original intention was to get UART communication between the Raspberry Pi and the PIC18F4550 so that the Pi would transmit processed traffic data (such as the number of vehicles passed by the Pi) to the PIC. But this arrangement was faced with a big challenge after facing the incompatibility between the two devices at logic level. The GPIO pin-out of the Raspberry Pi uses 3.3V logic, whereas the 18F4550 of the PIC18F4550 uses 5V logic. The direct interconnection between the two led to unreliable transmission of data and failure to communicate at times. We noticed that to interface the devices safely we needed an additional level shifter or voltage converter because direct connection might easily cause the Pi to break a GPIO pin.

2) UART Communication Conflict:

In order to overcome the logic level problem and simplify the data transfer process, we added Arduino Uno as mediating controller between Raspberry Pi and PIC18F4550. The principle was that the Pi would transmit the data to the Arduino via a USB connection, and then the Arduino would transmit those data to the PIC via its TX and RX connector. This has however created another problem whereby when the Arduino is connected to the Raspberry Pi via USB, the UART controller on the device is used by the serial communication with the Pi. Due to this, it was not able to access its hardware serial pins, at the same time, to communicate with the PIC in real time. This resource conflict of the UART led to lost or delayed transmission of data and therefore this setup was not very steady and reliable to be used in continuous communication.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

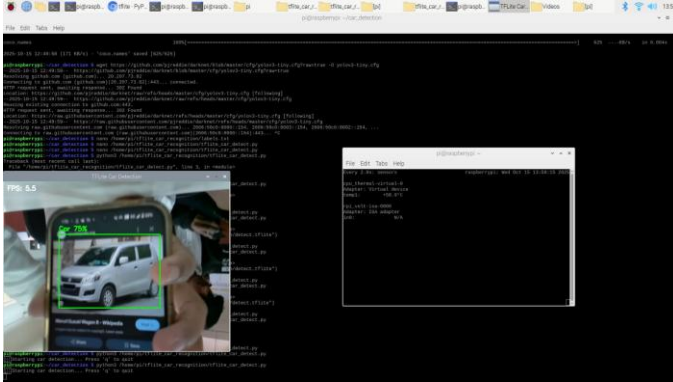


fig1: OpenCV Car detection



fig2: PIC18F4550 IR Sensor Testing

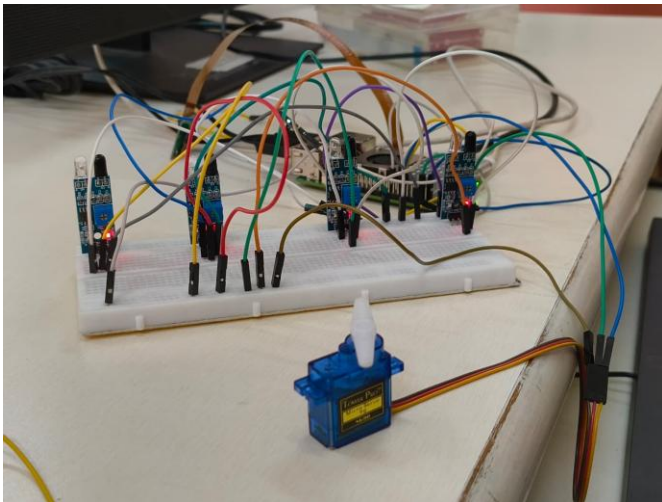


fig3: Raspberry PI Servo motor and IR sensor

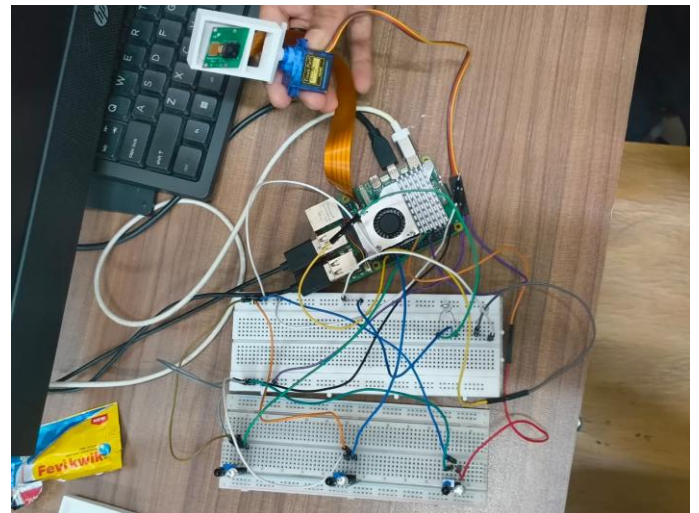


fig4: Final Circuit with Raspberry Pi



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Codes: Servo_IR Code

```
import RPi.GPIO as GPIO
import time

# --- GPIO Pin Setup ---
SERVO_PIN = 18
IR1_PIN = 17
IR2_PIN = 5
IR3_PIN = 22

# --- GPIO Initialization ---
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(SERVO_PIN, GPIO.OUT)
GPIO.setup(IR1_PIN, GPIO.IN)
GPIO.setup(IR2_PIN, GPIO.IN)
GPIO.setup(IR3_PIN, GPIO.IN)

# --- Servo Setup ---
servo = GPIO.PWM(SERVO_PIN, 50) # 50Hz PWM for servo

servo.start(0)

def set_servo_angle(angle):
    """Set servo position (0-180°)."""
    duty = 2 + (angle / 18) # Map angle to duty cycle
    GPIO.output(SERVO_PIN, True)
    servo.ChangeDutyCycle(duty)
    time.sleep(0.5)
    GPIO.output(SERVO_PIN, False)
    servo.ChangeDutyCycle(0)

# --- Main Loop ---
try:
    print("Starting Servo Control (Press Ctrl+C to stop)")

    last_angle = -1
```

```
while True:
    if GPIO.input(IR1_PIN) == 0:
        angle = 0
    elif GPIO.input(IR2_PIN) == 0:
        angle = 90
    elif GPIO.input(IR3_PIN) == 0:
        angle = 180
    else:
        angle = 0 # Default position if no IR detected

    if angle != last_angle:
        print(f"→ Moving Servo to {angle}°")
        set_servo_angle(angle)
        last_angle = angle
        time.sleep(0.1)

except KeyboardInterrupt:
    print("\nExiting program...")

finally:
    servo.stop()
    GPIO.cleanup()
```

OpenCV Code

```
import cv2
import numpy as np
import tf.lite_runtime.interpreter as tflite
import RPi.GPIO as GPIO

from time import time, sleep

# === GPIO Setup ===
RED_LED = 16
GREEN_LED = 26
GPIO.setmode(GPIO.BCM)
GPIO.setup(RED_LED, GPIO.OUT)
GPIO.setup(GREEN_LED, GPIO.OUT)
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
GPIO.output(RED_LED, GPIO.HIGH) # Start with RED
ON

GPIO.output(GREEN_LED, GPIO.LOW)

# === Load TFLite Model ===

MODEL_PATH =
"/home/kautik/tflite_car_recognition/detect.tflite"

LABEL_PATH =
"/home/kautik/tflite_car_recognition/labels.txt"

interpreter =
tflite.Interpreter(model_path=MODEL_PATH)

interpreter.allocate_tensors()

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
input_shape = input_details[0]['shape'][1:3]

# === Load labels ===

with open(LABEL_PATH, "r") as f:

    labels = [line.strip() for line in f.readlines()]

# === Initialize Camera ===

gst_pipeline = (

    "libcamerasrc ! video/x-
raw,width=640,height=480,framerate=30/1,format=
RGB !"

    "videoconvert ! appsink"

)

cap = cv2.VideoCapture(gst_pipeline,
cv2.CAP_GSTREAMER)

if not cap.isOpened():

    print("❌ Failed to open CSI camera.")

    GPIO.cleanup()

    exit(1)

print("✅ Starting Smart Traffic System (press 'q' to
quit)")

# === Helper function for LED control ===

def control_lights(car_count):

    if car_count == 0:
```

```
        return

    # Define green durations

    duration = {1: 3, 2: 6}.get(car_count, 10) # default
10 for >=3

    print(f"🚗 {car_count} car(s) detected → GREEN for
{duration}s")

    # Turn GREEN ON

    GPIO.output(RED_LED, GPIO.LOW)

    GPIO.output(GREEN_LED, GPIO.HIGH)

    sleep(duration)

    # Turn RED back ON

    GPIO.output(GREEN_LED, GPIO.LOW)

    GPIO.output(RED_LED, GPIO.HIGH)

    print("🔴 Back to RED, waiting for new input...\n")

# === Main Loop ===

while True:

    ret, frame = cap.read()

    if not ret or frame is None:

        print("⚠️ Frame capture failed. Retrying...")

        continue

    frame_bgr = cv2.cvtColor(frame,
cv2.COLOR_RGB2BGR)

    input_data = cv2.resize(frame, tuple(input_shape))

    input_data = np.expand_dims(input_data,
axis=0).astype(np.uint8)

    start_time = time()

    interpreter.set_tensor(input_details[0]['index'],
input_data)

    interpreter.invoke()

    boxes =
interpreter.get_tensor(output_details[0]['index'])[0]

    classes =
interpreter.get_tensor(output_details[1]['index'])[0]

    scores =
interpreter.get_tensor(output_details[2]['index'])[0]
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
car_count = 0

for i in range(len(scores)):
    if scores[i] > 0.5 and int(classes[i]) < len(labels):
        label_name = labels[int(classes[i]).lower()]
        if label_name == "car":
            car_count += 1
            ymin, xmin, ymax, xmax = boxes[i]
            left, top, right, bottom = int(xmin*640),
            int(ymin*480), int(xmax*640), int(ymax*480)
            cv2.rectangle(frame_bgr, (left, top), (right,
            bottom), (0, 255, 0), 2)
            car_count = min(car_count, 3)
            control_lights(car_count)
            fps = 1 / (time() - start_time)
            cv2.putText(frame_bgr, f"Cars: {car_count} FPS:
            {fps:.1f}", (10, 25),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255,
                255), 2)
            cv2.imshow("Smart Traffic - Car Detection",
            frame_bgr)
            if cv2.waitKey(1) & 0xFF == ord('q'):
                print("🔴 Quitting...")
                break
# === Cleanup ===
cap.release()
cv2.destroyAllWindows()
GPIO.cleanup()
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Applications:

- 1) Urban Traffic Management: This system is a good control mechanism that tries to control the traffic within the busy intersection by adjusting the timings of the signals depending on the real time vehicle density to minimize the congestion and the waiting time.
- 2) Emergency Vehicle Priority: It is capable of identifying emergency vehicles like ambulances, fire brigades, police cars giving them priority to clear traffic fast to increase the response time.
- 3) Pollution and Energy Reduction: Reduction of unnecessary vehicle idling, as well as increased green light duration, less fuel consumption, air pollution and energy use by traffic lights.
- 4) Prevention of Accident: Traffic flow- Any response to the density of traffic should be carried out properly, as this reduces the accident possibility.
- 5) Campus or private road management and smart cities: It can be used in big campuses, industrial regions, or even individual colonies to manage internal traffic in the most efficient way that can guarantee the efficient and safe movement of people within confined spaces and can be included in smart city development as a solution to automate with real time information.

Future Scope of Work:

- 1) Advance Vehicle Detection: Incorporation of machine learning models to categorize the type of vehicles like car buses, trucks two wheelers to improve traffic management.
- 2) Priority of emergency vehicles: ambulances fire brigades, police vehicles having GPS or RFID or siren recognition will automatically be identified and assigned priority.
- 3) Network Traffic Junctions: Intersections are connected in multiples to have city wide adaptive traffic control systems and real time optimization.
- 4) IoT and Cloud Integration: Support real time traffic monitoring and analytics data collection to support traffic planning.
- 5) Scalability and Smart City Application: a larger scale application in urban areas will cut down on congestion, enhance mobility and city planning.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Conclusion:

Our project on developing Smart Traffic Controller with the use of PIC18F4550 and Raspberry Pi demonstrates the application of intelligent, adjustable traffic control system to control the traffic at the intersection. This system demonstrates how it is possible to effectively address real world problem such as traffic congestion and inefficient signal timing, by combining IR sensors used to detect vehicles with camera-based Raspberry Pi with microcontrollers and sensors. The project demonstrates how electronics and algorithms can be combined to achieve an adjustable level of traffic management solutions, with possibilities of further developments in the application to smart cities.

References:

1) **Microchip PIC18F4550 Datasheet**

[PIC18F4550 | Microchip Technology](#)

– Provides technical specifications, pin configuration, and peripheral details for the PIC18F4550 microcontroller used for traffic signal control.

2) **Raspberry Pi 4 Model B Documentation**

<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>

– Official guide for setup, GPIO interfacing, and programming on Raspberry Pi, used for vehicle detection and image processing.

3) **Pi Camera Module V2 Documentation**

[Camera - Raspberry Pi Documentation](#)

– Technical specifications and interfacing guide for the camera module capturing traffic images.

4) **OpenCV Python Documentation**

[OpenCV: OpenCV-Python Tutorials](#)

– Comprehensive guide for image processing, contour detection, and real-time vehicle counting.

5) **UART Communication Protocol Guide**

[UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter | Analog Devices](#)

– Essential for understanding the underlying principles of UART communication, which is utilized for serial data transfer between the PIC18F4550 microcontroller and the Raspberry Pi in project.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

Appendix:

A.

Component Name	Quantity	Specification Details	Purpose
PIC18F4550 Microcontroller	1	• Operating Voltage: 4.2V–5.5V • Flash Memory: 32 KB • SRAM: 2 KB • EEPROM: 256 Bytes • Operating Frequency: up to 48 MHz • Communication: UART, SPI, I ² C, USB • I/O Pins: 35	Acts as the main controller that reads input from IR sensors, controls LEDs and servo motor, and communicates with Raspberry Pi.
IR Sensors	4	• Operating Voltage: 3.3V–5V DC • Detection Range: 2 cm–30 cm • Output: Digital (High/Low) • Response Time: < 500 μ s	Detects the presence or absence of vehicles at each road junction to determine traffic density.
Servo Motor (SG90 / MG995)	1	• Operating Voltage: 4.8V–6V • Rotation Angle: 0°–180° • Control Signal: PWM • Speed: 0.1 s/60° • Torque: 1.8–10 kg·cm (model dependent)	Rotates the camera to different road directions based on traffic congestion analysis.
Raspberry Pi 4 Model B	1	• Processor: Quad-core ARM Cortex-A72 @ 1.5 GHz • RAM: 2–4 GB LPDDR4 • Operating Voltage: 5V • GPIO Pins: 40 • Connectivity: USB, HDMI, Wi-Fi, Bluetooth, Ethernet • OS: Raspberry Pi OS	Processes live camera images, counts vehicles, and calculates adaptive green light duration using image processing algorithms.
Pi Camera Module V2	1	• Resolution: 8 MP • Video: 1080p @ 30 fps • Interface: CSI Connector • Lens: Fixed Focus • Power Supply: 3.3V	Captures real-time traffic images at the junction to monitor vehicle density for adaptive signal control.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune
